The MCP Trilogy, Part 1 — The Why: From ChatGPT to the Model Context Protocol

> Study notes on the first part of **The MCP Trilogy** by Nitish (*CampusX*). This video explains how AI usage evolved after ChatGPT, what problems that evolution created, and why the **Model Context Protocol (MCP)** became necessary. Explanations, diagrams, examples, and extra points have been added for learning.

Topic Model Context Protocol **Part 1 of 3 (The Why)** **Level Beginner**

Table of Contents

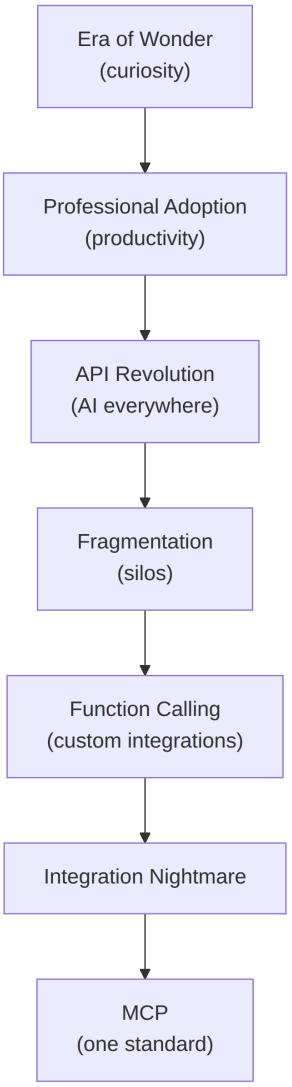
1. Overview
2. The Evolution of AI Interaction
3. The Problem of Context and Tools
4. Function Calling and the Integration Nightmare
5. The MCP Solution
6. Before vs After MCP: A Worked Example
7. Additional Insights
8. What MCP Does Not Solve
9. Key Takeaways
10. Self-Check Questions
11. Glossary
12. Resources

1. Overview

The video traces the journey from the public release of *ChatGPT* (November 2022) to today's AI ecosystem. The central argument is simple:

> **AI models became powerful quickly, but connecting them to the places where real work and data live stayed messy, manual, and expensive.**

MCP is presented as the answer to that gap: a single, standard way for AI applications to talk to external tools and data.



2. The Evolution of AI Interaction

2.1 The Era of Wonder (0:04:47)

After ChatGPT launched, most people used it out of **curiosity**:

- Writing poems, jokes, and stories
- Asking trivia and general-knowledge questions
- Testing what the model could and could not do

Added point: this phase mattered because it built **mass familiarity**. Millions of people learned the chat interface, which made later professional adoption much faster than with most new technologies.

Limitation of this phase: the model only knew its training data and whatever the user typed. It had no access to personal files, company data, or live information.

2.2 Professional Adoption (0:06:28)

People began using AI for real **productivity tasks**:

Task	Example
Summarizing	Turning a 40-page report into a one-page brief
Coding	Debugging an error message or explaining a function
Planning	Designing a course curriculum or project plan
Writing	Drafting emails, proposals, and documentation

Added point: in this phase the **user was the bridge**. To summarize a document, you had to copy it into the chat. To debug code, you pasted the code and the error. The AI was useful, but only as useful as the context you manually carried to it.

2.3 The API Revolution (0:08:09)

LLM providers opened their models through **APIs**, and software companies started building AI directly into their products:

- *Google Workspace* (Docs, Gmail, Sheets)
- *Microsoft 365* (Word, Outlook, Excel)
- Developer tools, note-taking apps, CRMs, chat tools

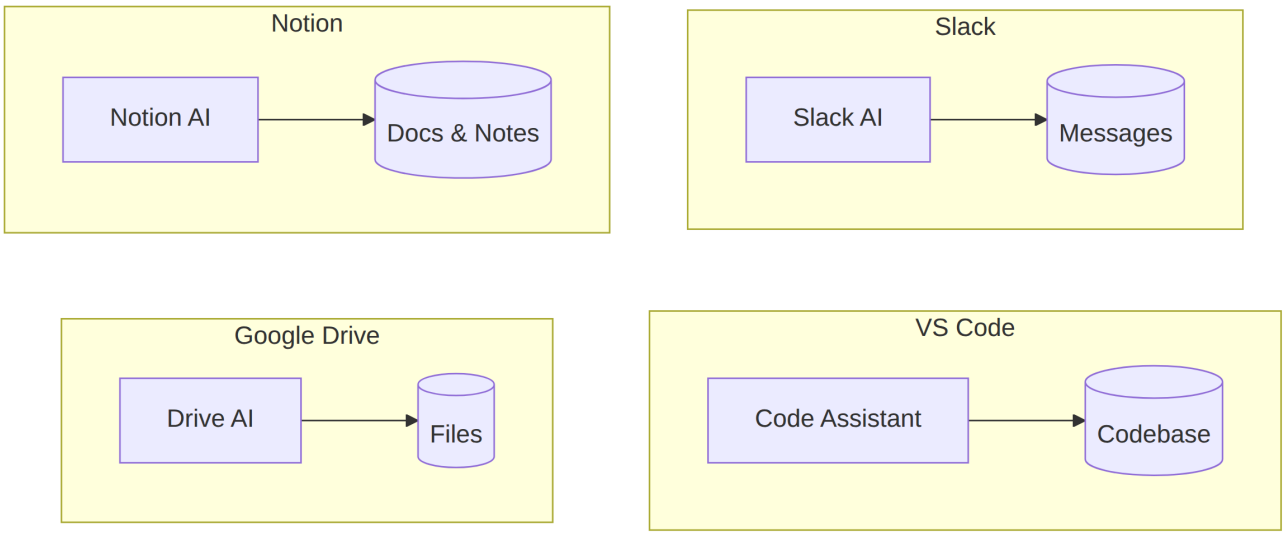
AI features became **ubiquitous**. But each product built its own AI, connected only to its own data. This created the next problem: **fragmentation**.

---

3. The Problem of Context and Tools

3.1 Fragmentation (0:09:48)

AI ended up living in separate **silos**:



Each assistant only sees its own data. None of them can answer a question like:

> *"Based on the bug reported in Slack yesterday and the spec in Notion, which file in the repo should I fix?"*

That question needs context from **three** systems, but no single AI can see all three.

Added point: real work is almost always **cross-tool**. A developer's task might start in a ticket, depend on a design document, and end in a code change and a team message. Siloed AI helps with each step but not with the workflow as a whole.

3.2 Context Assembly and the "Human API" (0:12:21, 0:18:21)

Because tools couldn't talk to each other, **humans** had to move the context:

1. Open Slack, copy the bug report
2. Open Notion, copy the relevant spec section
3. Open the code editor, copy the suspicious function
4. Paste everything into ChatGPT
5. Copy the answer back into the editor and Slack

The video calls this the **"human API"**: the person acts as the connector between systems.

Why this is bad:

Problem	Explanation
Slow	Minutes of copying for every question
Error-prone	Easy to miss the one piece of context that matters
Not scalable	Impossible to automate or repeat reliably
Stale	Pasted data is a snapshot; it doesn't update
Privacy risk	Sensitive data gets pasted into places it shouldn't go

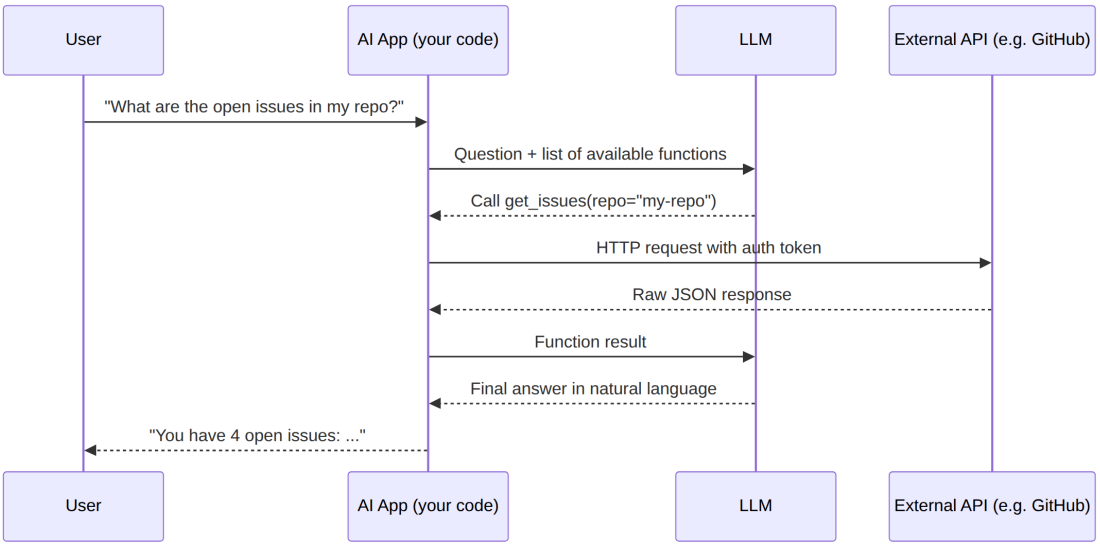
Added point: the quality of an LLM's answer depends heavily on the **context** it receives. A strong model with poor context gives poor answers. This is why the protocol is named *Model **Context** Protocol*: the core problem is getting the right context to the model.

---

4. Function Calling and the Integration Nightmare

4.1 What function calling is (0:21:20)

Function calling (also called **tool use**) lets an LLM request that an external function be run. The developer describes available functions, and the model decides when to call them.



Example of a function definition a developer writes for the model:

```
{
  "name": "get_issues",
  "description": "List open issues for a GitHub repository",
  "parameters": {
    "type": "object",
    "properties": {
      "repo": { "type": "string", "description": "Repository name" },
      "state": { "type": "string", "enum": ["open", "closed"] }
    },
    "required": ["repo"]
  }
}
```

This was a big step forward: the AI could now **fetch live data and take actions** instead of relying on copy-paste.

4.2 Why it became an "integration nightmare" (0:30:02–0:34:39)

Function calling works well for one or two tools. The trouble starts at scale. For **every** tool, in **every** AI app, the developer must write and maintain:

- Function schemas (names, descriptions, parameters)
- API client code (endpoints, pagination, rate limits)
- Authentication (OAuth flows, token storage, refresh)
- Data formatting (turning raw JSON into model-friendly text)
- Error handling and retries

Overhead	What it looks like in practice
Maintenance	An API changes its response format and your integration silently breaks
Cost	Engineering time spent on plumbing instead of product features
Security	API keys and tokens scattered across many codebases
Duplication	Ten companies each build their own GitHub integration
Lock-in	Function schemas written for one model provider don't transfer cleanly to another

Added point: before MCP, each model provider had a slightly different tool-calling format. So an integration was tied both to the **tool** and to the **AI provider**, doubling the lock-in.

---

5. The MCP Solution

5.1 What MCP is (0:36:10)

The **Model Context Protocol** is an open standard, introduced by Anthropic in November 2024, that defines a **common language** between:

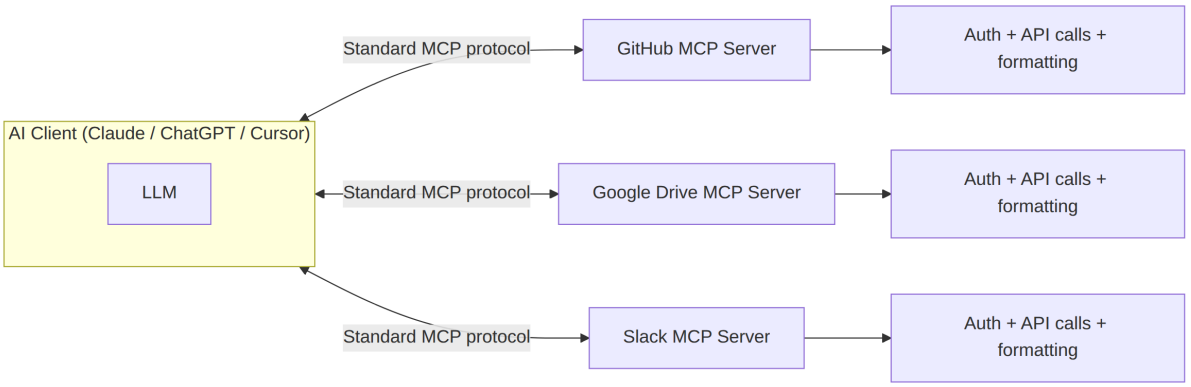
- **AI clients**: applications like *ChatGPT*, *Claude*, and *Cursor*
- **MCP servers**: connectors to data and tools like *GitHub* and *Google Drive*

Analogy: MCP is often compared to **USB-C**. A laptop doesn't need a special port for every device; it has a standard port, and every device that follows the standard just works.

Another useful analogy is **HTTP for the web**: browsers and websites don't need custom agreements with each other because they all speak HTTP.

5.2 How it works (0:41:26–0:42:00)

MCP moves the "heavy lifting" **to the server**:



Responsibility	Who handles it with MCP
Authentication with the service	MCP server
Calling the service's API	MCP server
Formatting data for the model	MCP server
Describing available tools	MCP server (the client discovers them automatically)
Deciding which tool to use	LLM in the client
Connecting to servers	Client, through a config entry, not custom code

The client doesn't need to know how GitHub's API works. It only needs to speak MCP.

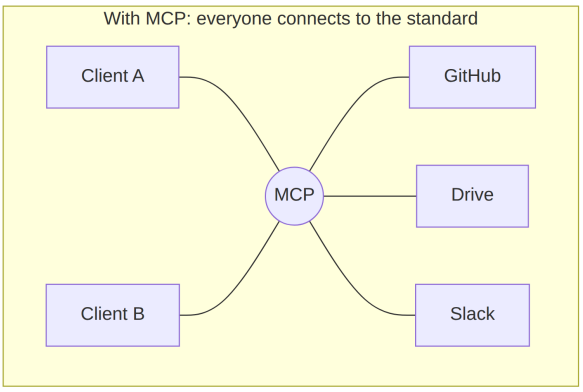
5.3 Benefits (0:43:26–0:47:28)

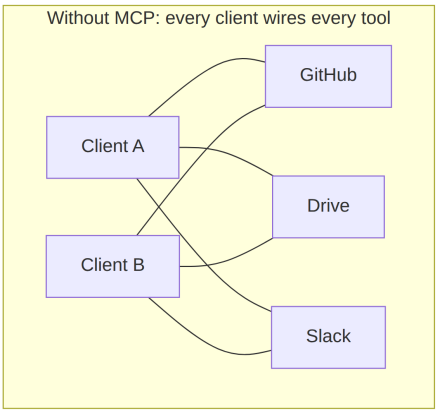
From n × m to n + m

Suppose there are **n** AI clients and **m** tools.

	Without MCP	With MCP
Formula	n × m integrations	n + m implementations
3 clients, 4 tools	**12**	**7**
10 clients, 50 tools	**500**	**60**
20 clients, 200 tools	**4,000**	**220**

The gap grows as the ecosystem grows, which is exactly why a standard pays off more over time.





Other benefits

- **Less maintenance:** when an API changes, only its MCP server needs updating, and every client benefits.
- **Lower cost:** teams reuse existing community servers instead of building integrations from scratch.
- **Better security:** credentials live in one place (the server's configuration) instead of being spread through application code.
- **Portability:** the same server works with Claude, Cursor, ChatGPT, or any other MCP-compatible client, so switching AI tools doesn't mean rebuilding integrations.
- **Ecosystem growth:** anyone can publish a server, so the number of available tools grows quickly.

---

6. Before vs After MCP: A Worked Example

Task: "*Summarize this week's open GitHub issues and save the summary to a Google Drive doc.*"

Before MCP (human API or custom code)

1. Developer writes a GitHub client with OAuth and pagination
2. Developer writes a Google Drive client with its own OAuth
3. Developer writes function schemas for both, in the AI provider's format
4. Developer writes orchestration code to pass results between the model and APIs
5. Developer maintains all of the above forever, and repeats it for any other AI app

After MCP

1. Add the GitHub MCP server and Google Drive MCP server to the client's config
2. Ask the question in plain language

```
{
  "mcpServers": {
    "github": { "command": "npx", "args": ["-y", "<github-mcp-server-package>"] },
    "gdrive": { "command": "npx", "args": ["-y", "<gdrive-mcp-server-package>"] }
  }
}
```

The client discovers the tools, the model picks the right ones, and the servers handle authentication and API details.

---

7. Additional Insights

1. **Standards win when ecosystems grow.** USB, HTTP, and SQL all succeeded because they reduced the number of custom agreements needed. MCP applies the same idea to AI tool access.
2. **MCP separates concerns.** Tool builders focus on great servers; AI app builders focus on great clients and models. Neither has to understand the other's internals.
3. **It enables agents.** AI agents need to use many tools across multi-step tasks. Without a standard, every agent framework reinvented tool integration. MCP gives agents a shared toolbox.
4. **Discovery is a hidden superpower.** With custom function calling, the developer hard-codes the tool list. With MCP, the client asks the server what it offers, so new tools can appear without changing client code.
5. **It's not only about tools.** MCP also standardizes **resources** (read-only data like files or records) and **prompts** (reusable templates), covered in Part 2 of the series.
6. **Adoption matters as much as design.** A protocol is only valuable if many clients and servers support it. MCP's value comes from broad adoption across AI apps, IDEs, and tool vendors.

---

8. What MCP Does Not Solve

To keep a balanced view, it's worth knowing the limits:

- **Model quality still matters.** MCP gives the model access to tools; it doesn't make the model choose them wisely.
- **Security risks move, they don't vanish.** A malicious or poorly written MCP server can misuse access. Only install servers from trusted sources.
- **Prompt injection.** Data fetched through MCP (a web page, an email, an issue comment) can contain hidden instructions aimed at the AI. Treat fetched content as data and keep humans in the loop for sensitive actions.

- **Context window limits.** Connecting many servers can overwhelm the model with tools and data. Fewer, well-described tools often work better.
- **Someone still writes the server.** The integration work doesn't disappear; it happens once in the server instead of many times in each client.
- **The standard is still evolving.** Features and transports have changed since launch, so always check the current specification.

---

9. Key Takeaways

- AI usage moved from **curiosity → productivity → embedded everywhere**.
- Embedding AI everywhere created **silos**: each AI only saw its own data.
- Users became the **"human API"**, manually copying context between tools.
- **Function calling** let AI reach external data, but building and maintaining custom integrations for every tool and app became an **integration nightmare**.
- **MCP** standardizes the connection: servers do the heavy lifting, clients just speak the protocol.
- The math changes from **n × m** to **n + m**, lowering maintenance, cost, and security overhead.

---

10. Self-Check Questions

1. What was the main limitation of AI during the "Era of Wonder"?
2. Why does the video describe users as a "human API"?
3. How did the API revolution lead to fragmentation?
4. What does function calling allow an LLM to do?
5. List three types of overhead that made function calling an "integration nightmare".
6. With 8 AI clients and 25 tools, how many integrations are needed without MCP? With MCP?
7. Which responsibilities move to the MCP server?
8. Name two problems MCP does **not** solve.

▼ Answer to Q6

Without MCP: 8 × 25 = **200**. With MCP: 8 + 25 = **33**.

---

11. Glossary

Term	Meaning
LLM	Large Language Model, e.g. the model behind ChatGPT or Claude
Context	The information given to a model to answer a question
Context assembly	Gathering relevant information from different sources for the model
Human API	A person manually moving data between tools
API	Application Programming Interface, a way for software to talk to other software
Fragmentation	AI features scattered across tools that don't share data
Function calling / tool use	An LLM requesting that an external function be run
MCP	Model Context Protocol, an open standard for connecting AI apps to tools and data
MCP client	The AI application side that connects to servers
MCP server	A program that exposes a tool or data source through MCP
n × m problem	The number of custom integrations needed when every client connects to every tool directly

12. Resources

- Official MCP documentation: https://modelcontextprotocol.io
- MCP GitHub organization (specification, SDKs, reference servers): https://github.com/modelcontextprotocol
- Anthropic's MCP announcement: https://www.anthropic.com/news/model-context-protocol
- CampusX YouTube channel (The MCP Trilogy series by Nitish)

Credits

Original video content by **Nitish (CampusX)**. These notes add explanations, diagrams, examples, and extra insights for learning purposes.